Apple II 게임 한글 패치 실전 매뉴얼

바이너리 분석, 텍스트 추출·압축, 6502 출력 루틴, 14×8 조합형 글꼴

Nox Archaist의 실제 디스크·6502·압축·조합형 글꼴 구조를 사례로, 다른 Apple II 게임에서도 같은 분석을 반복할 수 있도록 정리한 기술 매뉴얼입니다.

문서 버전 1.3 · 2026-09-09

이 문서는 Apple II 게임의 한글 패치를 실제 엔진 구조에 맞춰 설계하는 방법을 설명한다. 예시는 Nox Archaist에서 확인한 구조와 수치를 사용하지만, 특정 게임의 파일을 그대로 복사하는 방법이 아니라 분석 순서와 판정 방법을 전달하는 것을 목표로 한다.

한글 패치는 문자열을 바꾸는 작업으로 시작하지만, 실제 난점은 다음 네 계약을 동시에 지키는 데 있다.

1. 디스크 컨테이너가 파일을 어떤 블록에 저장하는가
2. 로더가 파일을 어느 주소와 뱅크에 올리는가
3. 6502 출력 루틴이 한 바이트를 어떤 의미로 해석하는가
4. 새 글자와 문장이 원래의 압축·종료·화면 이동 규칙을 그대로 통과하는가

따라서 분석 결과는 “어떤 영어 문장을 어떤 한국어로 바꿨다”에서 끝나지 않는다. 파일 오프셋, 메모리 주소, 평문 길이, 압축 payload, 화면 출력 경로, 회귀 검사 결과가 서로 연결되어야 한다.

1. 먼저 세 층을 분리한다

Apple II 게임에서 같은 데이터가 세 위치에 나타날 수 있다.

층	확인할 것	잘못 섞었을 때 생기는 문제
디스크	HDV/PO, ProDOS 카탈로그, EOF, 데이터 블록	파일 경계를 잘못 읽거나 다른 파일을 덮는다
실행 이미지	GAME, LOADER, 오버레이, FONT의 로드 주소	디스크 오프셋과 6502 주소를 혼동한다
실행 중 상태	main/AUX/언어카드 뱅크, 캐시, 화면 버퍼	정적 검색에 안 보이는 복사본을 놓친다

Nox의 실제 흐름은 다음과 같다.

HDV → ProDOS 카탈로그 → 파일 바이트 → LOADER.P의 로드 목록 → main/AUX 메모리 → COUT → HGR 화면

게임 화면에서 보인 바이트를 곧바로 HDV의 같은 위치로 역산하면 안 된다. 먼저 해당 바이트가 어느 파일의 어느 블록에 있고, 로더가 어느 주소에 복사하며, 실행 중 어느 루틴이 읽는지 확인한다.

2. 파일을 분석하는 방법

2.1 HDV를 먼저 ProDOS 파일 목록으로 바꾼다

ProDOS 블록은 512바이트다. 카탈로그에서 파일 엔트리의 EOF와 데이터 블록 체인을 얻은 다음, 파일의 논리 바이트를 재구성한다. raw image에서 문자열을 grep하는 방식은 카탈로그 경계와 파일의 홀수 EOF를 잃기 쉽다.

prodos_io.py의 핵심 흐름은 다음과 같다.

1. 카탈로그 블록을 순회해 경로, 파일 형식, EOF, key block을 읽는다.
2. seedling/sapling/tree index block을 따라 데이터 블록 순서를 복원한다.
3. EOF까지만 이어서 논리 파일을 만든다.
4. 파일 이름과 논리 오프셋을 HDV 블록 오프셋과 함께 기록한다.

이렇게 추출한 파일을 기준으로 GAME, LOADER.P, FONT1/FONT2, DATA.TLK.*, DATA.LOOK*, MAP 계열을 나눈다. 수정 대상은 파일 이름과 파일 오프셋으로 표현하고, 실행 분석에서는 별도로 로드 주소를 기록한다.

2.2 파일과 메모리 주소를 대응시킨다

Nox에서 확인한 대응은 다음과 같다.

파일	로드 주소	크기 또는 역할
GAME	$6000	13,824B. COUT 훅과 전투 메시지 포함

파일	로드 주소	크기 또는 역할
LOADER.P	$2000	10,514B. 부팅 매니페스트와 파일 로드 순서
SRTN.NPC.TALK	$AE00 오버레이	NPC 대화 압축 해제
FONT1/FONT2	AUX $5900	각 768B. 부팅 후 실제 글리프
DATA.TLK.*	파일 데이터	슬롯별 ZX7B 압축 대화

LOADER.P 내부의 파일명 목록만 읽어도 로드 순서를 알 수 있지만, 주소는 실행 코드의 로드 테이블과 동적 관찰로 다시 확인한다. 같은 파일 이름이 다른 시점에 다른 AUX 뱅크로 로드될 수도 있기 때문이다.

2.3 정적·동적·내용을 교차 확인한다

사용하지 않은 것처럼 보이는 영역을 곧바로 빈 공간으로 취급하지 않는다. 다음 세 검사를 모두 통과해야 한다.

1. 정적 참조: 역어셈블리에서 해당 주소를 읽거나 쓰는 명령이 없는가
2. 동적 접근: 부팅·맵 이동·전투·상점·대화·세이브 시나리오에서 read/write가 없는가
3. 내용 판정: 실제 바이트가 코드·문자열·테이블·화면 데이터가 아닌가

Nox에서는 main $7600이 짧은 이동 시험에서는 읽히지 않았지만 전투 메시지 영역이었다. AUX $1600도 일부 하니스에서는 조용해 보였지만 전체 시나리오에서 576회의 write가 발생했다. 두 영역 모두 코드나 폰트를 배치할 수 없었다. “미접근”은 “미사용”의 증거가 아니다.

3. 텍스트를 추출하고 의미를 복원한다

3.1 먼저 후보를 넓게 찾고, 이후 정본 파서로 좁힌다

초기 조사는 batch_extract.py와 zx7b_scan.py로 DATA.TLK.*, DATA.LOOK*, TEXT.MISC, CWSPEAK, NPC.JOIN*를 훑는다. 이 단계의 목적은 파일별로 압축 스트림 후보, 시작·끝 오프셋, 확장된 평문 길이를 얻는 것이다.

스캔 결과를 사람이 읽을 수 있게 바꿀 때의 규칙은 다음과 같다.

- hi-ASCII 인쇄 바이트는 하위 7비트 문자로 표시한다.
- 0x8D는 줄바꿈으로 표시한다.
- 0x00은 레코드 경계인 <0>으로 표시한다.
- 해석하지 않은 바이트는 <xx> 형식으로 남긴다.
- 파일·세그먼트·시작·끝·압축 길이·평문 길이를 manifest에 기록한다.

이 덤프는 조사 지도이지 재주입 원문이 아니다. <xx>를 임의로 삭제하거나 모든 바이트를 ASCII로 간주하면 마커와 종료 바이트를 잃는다.

3.2 DATA.TLK는 슬롯 인덱스로 파싱한다

Nox의 정식 컨테이너 형식은 다음과 같다.

entry = slot 1B + compressed length u16 little-endian + ZX7B stream

슬롯 바이트는 NPC 순번×8로 증가한다. 파일 뒤에는 00 FF 종결 또는 게임이 읽지 않는 tail이 올 수 있다. tlk_container.py는 슬롯 증가와 payload 길이를 동시에 확인하므로, 무작위 압축 후보를 잡는 brute scan보다 안정적이다.

L001의 slot A8에는 80 00 00 헤더와 헤더리스 wizard 스트림이라는 예외가 있었다. 이런 특례는 일반 규칙을 바꾸는 근거가 아니라, 원본에서 확인된 예외 목록으로 분리해 기록한다.

3.3 레코드에서 마커와 번역 본문을 분리한다

tlk_workbook.py는 한 슬롯의 평문을 00으로 나누고, 각 레코드 앞부분에서 구조 마커를 분리한다. 마커부는 첫 0xAE에서 끝난다. $I 외형, $!KEYWORD, $N/$Y/$W/$A 응답 모드, $K 키워드, $S·$P·$H·$T 조건 마커는 번역하지 않는다.

본문을 편집용 텍스트로 바꿀 때는 다음 왕복 규칙을 사용한다.

- 0x8D ↔ 리터럴 줄바꿈
- 0xBC ↔ <bc>
- 일반 hi-ASCII 문자 ↔ 하위 7비트 ASCII
- 나머지 바이트 ↔ <xx>

JSONL 워크북의 최소 행은 파일, slot, record, 지역, NPC 추정명, 원문 본문, 번역 본문, prefix_hex다. 번역문을 만들 때 prefix_hex와 record 인덱스를 그대로 보존하면, 같은 문장이 키워드 체인에서 어느 위치에 있는지 잃지 않는다.

3.4 고정 길이 테이블은 별도로 추출한다

DATA.TLK의 압축 레코드와 MAP 이름·동료 이름·아이템명은 같은 방식으로 다루지 않는다. 고정 필드는 위치와 용량이 의미의 일부다.

실제 사례:

- NPC.JOIN8의 Cutter 필드는 6B라서 커터 4B와 NUL 2B가 들어간다.
- Blade 필드는 5B라 블레이드 8B를 넣을 수 없으므로 다른 표기나 원문 유지가 필요하다.
- DATA.MAP.TOWN의 Ghrodwir 필드는 8B, Ghrodwir Mines 필드는 14B다.
- Liches와 Stealing! Proceed?는 각각 다른 파일과 길이 제약을 가진다.

이런 이름을 대사 파일에서 찾지 못했다고 전체 문자열 파일을 다시 압축하면, 실제 화면에서 쓰는 지도 레이어와 인벤토리 레이어를 놓치게 된다. MAP, NPC 고정 필드, 대사, LOOK 결과를 각각 추출하고 각각의 용량·널 경계를 기록한다.

4. 압축과 재조립을 분석하는 방법

4.1 ZX7B는 앞에서 읽는 압축이 아니다

Nox의 ZX7B 루틴은 파일 뒤에서 앞으로 입력을 소비하고, 출력도 뒤에서 앞으로 만든다. zx7b.py는 다음을 모델링한다.

- 첫 literal 바이트
- flag bit 0 literal / 1 match
- interlaced Elias gamma 길이
- 짧은 거리와 확장 거리
- 16비트 overflow를 EOF 표식으로 사용

이 구조에서는 압축 payload가 짧아졌다는 사실만으로 안전하지 않다. 뒤에서 진행하는 출력 포인터가 아직 읽지 않은 입력 바이트를 덮을 수 있기 때문이다.

4.2 제자리 압축의 최소 여유를 계산한다

게임은 payload를 $99F6 근처에 두고 평문을 $9A00에 만든다. payload 앞의 2B는 평문 길이 헤더로 사용되고, 평문 버퍼는 $9A00부터 $ADFC까지다. payload와 출력 끝의 초기 간격을 계산한 뒤, 압축 비트스트림을 실제 순서로 재생하며 다음 값을 추적한다.

minimum margin = initial gap − (지금까지 출력한 바이트 − 지금까지 소비한 입력 바이트)

이 값이 음수면 별도 버퍼에서 왕복 압축이 성공해도 게임 루틴에서는 실패한다. 에버튼 주민 대화의 기존 payload는 평문 34B에 45B였지만 최소 여유가 -1B였다. 첫 FC 출력이 아직 읽지 않은 flag를 덮어 대화가 text mode로 바뀌었다. payload를 41B로 만들고 EOF fill bit와 2B 반복을 조정해 실제 루틴에서 +3B 여유를 확보했다.

4.3 재조립은 세 번 검증한다

tlk_rebuild.py의 검증 순서는 다음과 같다.

1. 워크북에서 prefix·본문·레코드 구분·suffix를 다시 조립한다.

2. 한글은 ko_johab.py로 2바이트화하고 ASCII·마커·줄바꿈은 별도 규칙으로 보존한다.
3. zx7b_compress.py로 payload를 만들고 게임과 같은 zx7b.py로 다시 푼다.
4. 평문이 원본 또는 번역 레이어와 바이트 단위로 일치하는지 확인한다.
5. payload 길이, 평문 길이, $1400 상한, 최소 여유를 검사한다.
6. entry 슬롯·EOF·종결 tail을 다시 파싱한다.

Nox 전체 31개 DATA.TLK, 444개 스트림에서 여유가 음수인 후보 88개 중 84개만 안전 조건을 만족했다. 나머지는 문장을 억지로 줄이거나 파일을 확장하지 않고 보류했다. 전수 검사는 적용 명령이 아니라 위험 후보를 찾는 단계다.

5. 한글 바이트와 출력 루틴

5.1 기존 문자와 충돌하지 않는 2바이트 체계를 만든다

Nox에서 0xA0–0xFF는 이미 hi-ASCII 인쇄 문자로 쓰인다. 0x00은 레코드 구분, 0x8D는 CR, 0xFC/0xFF는 엔트리 종결, 0xA4/$·0xA5/%·0xA1/!은 $K·$!·%END 마커에 쓰인다. 0xA0은 WWPRINT가 단어 분리용으로 직접 비교한다.

따라서 양쪽 바이트에서 다음 값을 피한다.

00, 0D, 8D, 21, A1, 24, A4, 25, A5, A0, FC, FF

리드 풀 P1은 0x01–0x0C, 0x0E–0x1F, 0x80–0x8C, 0x8E–0x9F의 안전 구간에서 57개를 사용한다. 트레일 풀 P2는 금지 값을 제외한 오름차순 배열에서 196개를 사용한다.

초성 c(0–18), 중성 v(0–20), 종성 t(0–27, 없음=0)은 다음처럼 패킹한다.

- b1 = P1[c×3 + v÷7]
- b2 = P2[(v mod 7)×28 + t]

P1과 P2의 값이 일부 겹쳐도 된다. 리드 위치에서는 T1, pending 상태의 트레일 위치에서는 T2만 적용하는 상태 기계가 있기 때문이다.

5.2 COUT에서 한글 한 글자를 두 셀로 출력한다

원래 COUT 경로는 $7998에서 $7966 글리프 스테이징과 $0307 HGR 블리터를 호출하고, 한 셀을 그린 뒤 커서를 한 칸 전진한다. $7966은 AUX $5900의 8B 글리프를 $22F8에 복사한다. PRINT.STR은 $7AA1, WWPRINT는 $7CD9에 있다.

한글 훅은 다음 상태를 추가한다.

1. 리드 바이트를 받으면 초성·중성 상위 인덱스를 pending에 저장하고 화면에는 그리지 않는다.
2. 트레일 바이트에서 중성 하위·종성을 얻는다.
3. 세 마스크를 14×8 버퍼에 OR한다.
4. 왼쪽 7픽셀 8행을 그린 뒤 오른쪽 7픽셀 8행을 그린다.
5. 커서를 두 셀만큼 전진하고 pending을 지운다.

CR, NUL, 레코드 경계에서 pending을 반드시 지워야 ASCII와 한글이 섞인 다음 문장이 이전 글자의 절반과 조합되지 않는다.

6. 왜 14×8 글꼴인가

6.1 원본 글리프 단위에서 역산한다

Nox의 원래 FONT1/FONT2는 AUX $5900에 각각 768B로 올라가고, COUT의 $7966 루틴은 글리프 하나를 8B 복사한다. 즉 원래 화면 글리프는 8행의 1셀 단위이며, 실제 유효 폭은 7픽셀이고 남은 비트는 셀 간 여백과 하이레즈 색 번짐에 사용된다.

한국어 음절을 한 셀에 넣으면 다음 문제가 생긴다.

- 초성·중성·종성을 동시에 구분할 가로 공간이 부족하다.
- ㅐ·ㅒ·ㅖ·ㅙ처럼 두 기둥을 가진 모음이 한 줄로 뭉친다.

- 받침을 넣는 순간 위쪽 획과 아래쪽 획이 붙는다.
- COUT의 1셀 폭과 실제 한글 획 폭이 맞지 않는다.

두 셀을 이어 쓰면 실제로 사용할 수 있는 폭이 7+7=14픽셀이 되고, 세로는 원래 HGR 글리프와 같은 8행을 유지할 수 있다. 이 때문에 14×8을 선택했다. 16B가 필요한 이유도 한 음절의 8행을 왼쪽 1B와 오른쪽 1B로 저장하기 때문이다.

6.2 14×8을 만드는 데이터 구조

조합기는 완성 음절 그림을 11,172개 저장하지 않는다. 다음 세 종류의 마스크를 저장하고 조합한다.

- 초성 19개와 받침 유무별 소형 폼
- 중성 21개와 세로·가로·복합·쌍기둥 문맥 폼
- 종성 28개와 2행·3행 받침 폼

14×8 행을 14비트로 만든 뒤, 각 행을 다음처럼 패킹한다.

- left7 = row & 0x7F
- right7 = row >> 7
- 8행×2B = 음절당 16B

저장 비트 0은 각 셀의 왼쪽 픽셀이다. 세로획과 대각선은 인접한 2픽셀을 켜서 Apple II 하이레즈의 색 번짐에서 끊기지 않도록 했다.

6.3 문맥 뱅크와 선택자를 분리한다

같은 ㄱ이나 ㅗ를 모든 음절에 똑같이 넣으면 받침이 있는 글자에서 위아래 간격이 무너진다. 따라서 중성 종류와 종성 유무에 따라 초성·중성·종성의 박스를 바꾼다.

현재 데이터 구조의 실측 규모는 325개 마스크, 49개 문맥 뱅크, 105개 음절 선택 예외다. compose.py는 유니코드 음절을 c/v/t로 분해하고, 문맥 뱅크를 고른 뒤 예외 선택자를 적용한다.

유니코드 분해는 다음으로 충분하다.

- n = codepoint − 0xAC00
- c = n ÷ 588
- v = (n mod 588) ÷ 28
- t = n mod 28

예외 선택자는 완성형 그림을 덮어쓰는 표가 아니다. 공유 자모 중 어떤 마스크를 선택할지 지정하는 표다. 그래서 예외 한 줄을 추가하면 같은 초성·중성·종성을 공유하는 다른 음절의 전후 출력도 함께 검사해야 한다.

6.3.1 Galmuri7 초안에서 DKB 조합형 기반으로 전환한 이유

초기 초안은 Galmuri7의 7×7 완성형 음절을 가로 2배로 확장한 뒤, 게임이 재사용할 수 있는 자모 마스크로 역분해하는 방식이었다. 이 방법은 갈무리7의 완성형 실루엣을 가까이 따라갈 수 있지만, 이미 합쳐진 글자에서 초성·중성·종성의 경계를 거꾸로 추정해야 한다는 문제가 있다. 한 마스크를 여러 문맥에서 공유해야 하므로 `난`의 ㄴ, `광`의 ㄱ, `쪽`의 종성처럼 서로 다른 음절에서 같은 마스크가 충돌하기 쉽다. 이 실험이 v7이다.

v7의 실제 화면 검수에서 `배`, `서`, `아무 키`, `교단`처럼 갈무리7과 비슷해야 하는 글자가 문맥에 따라 무너지는 것이 확인되었다. 따라서 완성형 글자를 계속 쪼개는 대신, 처음부터 자모 조합을 전제로 만든 DKB 8×4×4 자료를 v8의 출발점으로 바꾸었다. 여기서 DKB는 조합 원리만을 뜻하는 이름이 아니라, 한글도깨비 계열의 실제 자모 픽셀 원본과 선택 구조를 함께 가리킨다.

DKB 원본은 초성 8벌·중성 4벌·종성 4벌의 문맥 뱅크를 가진다. 각 음절에서 유니코드 초성·중성·종성을 계산한 뒤, 중성의 가로·세로 형태와 종성 유무에 맞는 뱅크를 선택한다. 16×16 원본 셀에서 실제 획을 조합하고, 활성 14행을 두 행씩 OR 접어 7개의 유효 스캔라인으로 만든 뒤 가로 16픽셀을 Apple II의 14열로 접었다. 마지막 행은 셀 간 간격으로 남겨 14×8 런타임 글리프에 넣었다. 이 변환은 갈무리7 완성형을 그대로 복사하는 것과 달리, 자모가 들어갈 허용 영역과 문맥을 처음부터 보존한다.

v9부터는 이 DKB 뱅크와 선택 구조를 유지한 채 Galmuri7 완성형을 목표 실루엣으로 삼아 픽셀 오차를 줄였다. 이후 v10~v12와 최종 수정에서는 사용 빈도가 높은 음절, 공유 마스크 회귀, 사용자가 지정한 보존 글자를 함께 고려했다. 따라서 최종 글꼴은 `Galmuri7의 시각적 기준`과 `DKB의 조합형 자모 구조`를 결합한 하이브리드다. 공유용 자료에는 게임 파일이나 DKB 원본 이미지를 포함하지 않고, 변환·검증이 끝난 마스크와 조합기만 둔다.

6.4 갈무리 참고 글꼴이 그대로 조합되지 않는 이유

갈무리 이미지의 한 글자는 이미 초성·중성·종성이 합쳐진 완성형 그림이다. Nox는 세 마스크를 OR해서 14×8에 배치하므로, 같은 픽셀이라도 다음 조건에 따라 의미가 달라진다.

- 초성 문맥이 세로모음용인지 가로모음용인지
- 종성 때문에 초성·중성 박스가 6행에서 3행으로 줄었는지
- ㅐ 계열의 두 기둥을 7픽셀 셀 안에 넣었는지
- 2픽셀 획이 색 번짐으로 이어지는지
- 오른쪽 7픽셀 셀의 시작 열이 한 칸 밀렸는지

갈무리의 ㅐ를 그대로 중성 슬롯에 복사하면 ㅣ처럼 보이고, 갈무리의 ㄱ 외곽을 그대로 초성에 넣으면 광·굉에서 ㅗ와 충돌한다. 참고 이미지는 목표 실루엣으로 사용하고, Nox의 허용 박스 안에서 마스크를 다시 분해해야 한다.

6.5 실제 마스크 수정의 분석 절차

문제가 된 완성 음절을 통째로 지우지 않고 다음 순서로 분해한다.

1. 유니코드 음절을 c/v/t로 분해한다.
2. 현재 선택된 초성·중성·종성 마스크를 각각 단독 렌더링한다.
3. 세 마스크의 OR 결과와 참고 도트를 행·열 좌표로 비교한다.
4. 차이가 한 마스크에만 있는지, 겹침으로 생겼는지 판정한다.
5. 그 마스크를 공유하는 음절 목록을 다시 생성한다.
6. 변경 전후 golden을 비교하고, 공유 음절의 회귀를 분류한다.

대표 사례:

음절	원인 마스크	수정 원칙
광·굉	ㄱ 초성 또는 ㅗ 계열 중성	초성과 ㅗ를 분리해 수직 획·윗돌출 위치를 각각 조정
쪽·쭉	쪽은 종성, 쭉은 중성	완성 글자를 덮지 않고 원인 마스크만 이동
찌	ㅉ 초성 오른쪽 열	가장 오른쪽 1픽셀을 잘라 공유 ㅉ의 폭 회복
찔	ㅉ·ㅣ·ㄹ	중성·종성 이동, 초성 윗줄 끝점 추가, 최종 종성 위치 재조정
몰	ㅁ 초성	초성 높이 4행을 2행으로 줄여 중간 빈틈 제거
윈	ㅟ 중성	아래 돌출점 x2–3을 x4–5로 이동하고 기존 선택자 유지
못	ㅁ·ㅗ·ㅅ	현재 조합이 정상인 기준 음절로 고정하고 불필요한 공유 마스크 변경을 하지 않음

윈 수정은 새 음절 그림을 만들지 않고 기존 V313 선택자를 유지했다. 윈과 쉿·뒷·월·쥔·쉭·퀵·휩·귓·뀐·쉴·쉼·휙을 함께 재렌더링해 다른 11,171음절이 바뀌지 않았고, 동일 화소 쌍도 0이었다.

7. 글꼴 파일을 만드는 방법

7.1 소스 마스크와 런타임 뱅크를 나눈다

편집용 font.json에는 자모 마스크, 문맥 박스, 뱅크 인덱스, 완성 음절 예외를 둔다. 런타임용 FONT 뱅크에는 이미 선택된 8행 글리프와 6502가 사용할 인덱스만 둔다. 이 둘을 섞으면 픽셀 수정과 디스크 배치 수정이 서로 영향을 준다.

kp_jamo_font.py의 기본 렌더러는 CHO/JUNG/JONG와 소형 폼을 만들고, compose 단계에서 14×8 OR 조합을 수행한다. export 단계에서는 뱅크당 고정 슬롯을 채우고, 빈 슬롯도 0으로 명시한다.

7.2 후보 비교표를 만든다

후보 하나마다 다음 열을 가진 비교표를 만든다.

음절 | c/v/t | 이전 마스크 ID | 새 마스크 ID | 이전 16B | 새 16B | 차이 픽셀 | 공유 음절 수 | 판정

판정은 세 가지로 나눈다.

- 마스크 원인과 수정 위치가 일치하고 공유 음절 회귀가 없음
- 차이는 줄었지만 공유 음절에서 추가 검토 필요
- 참고 이미지와 달라졌거나 허용 박스를 침범하여 보류

자동 픽셀 점수만으로 승인하지 않는다. Apple II 화면에서는 인접 픽셀 위치에 따라 색 줄무늬와 획 연결이 달라지므로, 대표 음절 시트와 실제 게임 화면을 함께 확인한다.

7.3 전수 golden과 CPU 검사를 분리한다

U+AC00부터 U+D7A3까지 11,172자를 순서대로 조합해 음절당 16B golden을 만든다. golden은 조합기가 같은 입력에서 같은 픽셀을 내는지 검사하는 기준이다.

그 다음 6502 COUT 경로에서 리드·트레일·CR·ASCII·pending 초기화를 실행한다. Nox 106차 검사는 221,380,508개 명령을 통과했고 글리프당 최대 실행량은 29,705였다. 이 수치는 코드 경로와 메모리 계약을 검증하지만, 실제 색 번짐·게임 속도·음악 IRQ 승인을 대신하지 않는다.

8. 패치 위치와 런타임 메모리

GAME의 $7998은 COUT 진입점이라 파일 내부 혹은 GAME 파일의 해당 오프셋에 둔다. FONT1/FONT2는 AUX $5900에 상주하므로, 새 마스크 뱅크를 놓을 때는 로더와 다른 오버레이가 덮지 않는지 확인한다.

메모리 영역을 고를 때는 다음을 기록한다.

- 정적 코드 참조 수
- 시나리오별 read/write 횟수
- 해당 영역의 기존 문자열·테이블·화면 데이터
- 로더가 다시 쓰는 시점
- 언어카드 뱅크 전환 시 동일 주소의 의미

한 번 읽기 전용으로 보인 폰트 캐시는 다음 화면에서 교체될 수 있고, 전투 메시지 영역은 전투를 실행해야만 읽힐 수 있다. 패치 코드는 주소 하나보다 로드 순서와 overwrite 시점을 함께 봐야 한다.

9. 번역 레이어와 빌드 게이트

번역은 추출 데이터에 직접 덮어쓰지 않고, 원문과 번역문을 같은 행에 둔 레이어로 만든다. 한 행에는 파일·slot·record·화자 추정·키워드·원문·번역문·prefix_hex·원문 바이트 길이·번역 바이트 길이를 둔다.

빌드 게이트는 다음 순서다.

1. 마커·토큰·대괄호·리터럴 CR·NUL·suffix 보존
2. 한글 2바이트 인코딩과 고정 필드 용량 검사
3. TLK 평문 재조립과 원문/번역 대상 비교
4. ZX7B 압축 및 제자리 minimum margin 검사
5. ProDOS 파일 되읽기와 변경 파일 목록 검사
6. COUT/6502 golden과 런타임 카나리아

7 대표 세이브·대화·MAP·인벤토리·전투 경로

자동 게이트가 통과해도 실기에서 화면·음악·입력 복귀를 확인하지 못한 경로는 미검증으로 표시한다.

10. 고장 사례를 원인별로 분리한다

10.1 대화 화면이 기계어처럼 변한 경우

원본과 패치 이미지에 같은 세이브·같은 입력을 넣는다. 원본이 정상 복귀하고 패치만 text mode로 전환되면 압축 payload·종료 경계·한글 바이트를 우선 조사한다. 에버튼 사례는 폰트 문제가 아니라 L005 두 슬롯의 제자리 압축 여유 -1B가 원인이었다.

10.2 음악과 화면이 함께 멈춘 경우

마지막 출력 문자열, 화면 전환 여부, 음악이 끊긴 시점, IRQ 실행 여부를 각각 기록한다. 문자열이 정상으로 보였다는 사실만으로 대화 루틴이 끝났다고 판단하지 않는다. 원본도 같은 입력에서 멈추면 패치 회귀가 아니며, 강제 하니스에서만 BRK가 난 결과도 실제 플레이 회귀로 사용하지 않는다.

10.3 지도에서 보이지 않는 장애물

같은 좌표와 세이브를 원본·패치판에 넣고 LOOK 결과, 이동 거부 문구, 객체 인덱스·바이트를 비교한다. 두 이미지가 같은 "풀"과 북쪽 이동 거부를 내면 MAP 번역이나 글꼴 문제로 수정하지 않는다.

10.4 토큰이 화면에 남는 경우

$K/$k, $!KEYWORD, %END는 화면 문장이 아니라 입력·응답·종료를 연결하는 토큰이다. 검색·추출 과정에서 선두 토큰을 0 sentinel로 간주하거나 번역문에서 제거하면 토큰이 그대로 보이거나 다음 응답으로 넘어가지 않는다. 토큰을 유지한 채 본문과 키워드 체인을 함께 검사한다.

11. 재현 가능한 작업 순서

1 ProDOS 카탈로그와 파일↔로드 주소 표를 만든다.
2 정적 참조·동적 접근·파일 내용으로 메모리 영역을 분류한다.
3 압축 후보를 스캔하고, 확인된 컨테이너는 슬롯 파서로 다시 읽는다.
4 마커와 본문을 분리한 JSONL 워크북을 만든다.
5 원문 왕복으로 추출·재조립 파서를 먼저 검증한다.
6 한글 바이트 코덱과 CR/NUL/pending 경로를 단독 검증한다.
7 14×8 마스크를 조합하고 공유 음절 비교표를 만든다.
8 golden·압축 margin·6502 COUT·ProDOS 되읽기를 통과시킨다.
9 대화·지도·인벤토리·전투에서 원본과 패치판의 동일 입력을 비교한다.
10 자동 결과, 에뮬레이터 결과, 실제 화면 결과를 서로 다른 판정으로 기록한다.

이 순서를 지키면 "문장을 바꾸었더니 다운되었다", "한 글자를 고쳤더니 다른 글자가 깨졌다", "참고 폰트를 넣었는데 모음이 II처럼 보인다"와 같은 증상을 각각 파일 경계, 압축 방향, 공유 마스크, 셀 폭 문제로 분리할 수 있다.

12. 핵심 공식과 수치

항목	값 또는 공식
음절 분해	n=codepoint−AC00, c=n÷588, v=(n mod 588)÷28, t=n mod 28
한글 패킹	b1=P1[c×3+v÷7], b2=P2[(v mod 7)×28+t]

항목	값 또는 공식
한글 픽셀	14×8, 8행×(left7+right7)=16B
TLK entry	slot 1B + clen u16 LE + ZX7B payload
제자리 여유	initial gap − (produced − consumed), 최소값이 0 이상이어야 함
평문 버퍼	$9A00–$ADFC, 최대 5,117B
COUT 경로	$7998 → $7966 → $0307
실제 글꼴 위치	AUX $5900, 8B 원본 글리프
조합 데이터	325 마스크, 49 뱅크, 105 선택자, 11,172자 golden

이 수치들은 Nox에서 확인한 한 구현의 기준이다. 다른 Apple II 게임에서는 같은 분석을 반복해 인코딩, 화면 폭, 메모리 뱅크, 출력 루틴, 압축·종료 규칙을 새로 확정해야 한다.